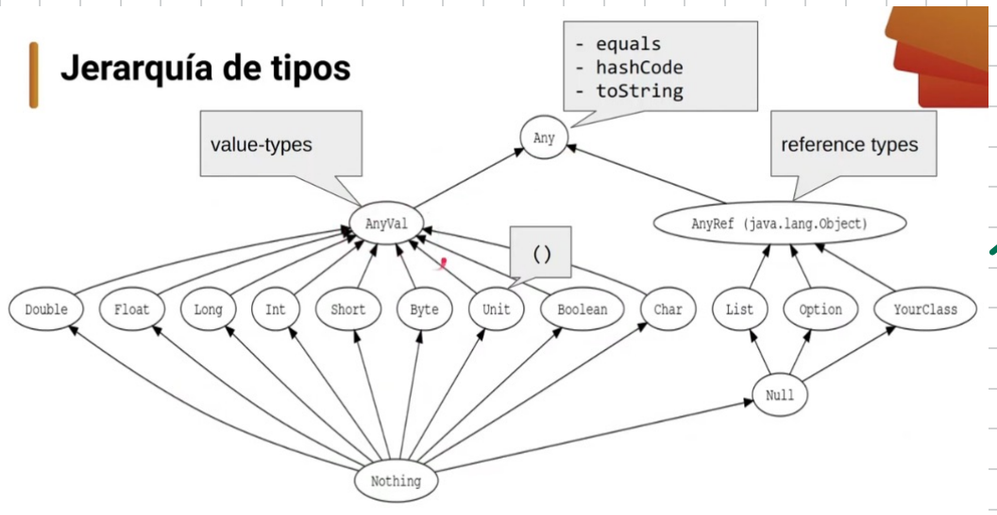
• Inferencia de Tipos.

Sabemos que si al definir una variable no declaramos el Tipo, Scala va a inferir el Tipo. Pero si ponemos una lista con varios Tipos, ¿Qué Tipo infiere Scala? para eso hay que entender la jerarquía de Tipos:



↑ Clase superior

Scala infiere que la lista es del Tipo que es superior a Todos los elementos de la lista.

Ej:

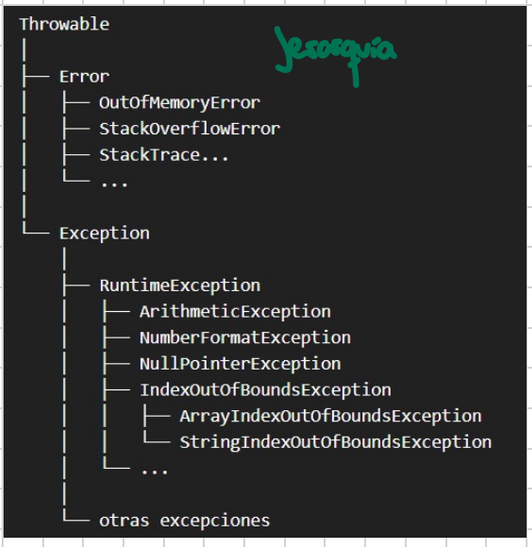
- Todos value types:

```
scala> val listAnyVal = List(12, true, 2.0, 'c')
listAnyVal: List[AnyVal] = List(12, true, 2.0, c)
```

- value types y reference types:

```
scala> val listAny = List(12, true, 2.0, 'c', "keepCoding")
listAny: List[Any] = List(12, true, 2.0, c, keepCoding)
```

• Excepciones y errores.



Throwable: Es la clase de Todo lo que puede ser lanzado (Throw) y capturado (catch) en Scala.

- Excepción: Es una situación anormal que ocurre mientras el programa se está ejecutando pero que el programa puede manejarla (No es Tan Terrible)
- Error: Es lo mismo que una excepción pero el programa no puede manejarla (Terrible)

• Throw.

Es una forma de lanzar una excepción, que hay que interruptir el programa y mostrar un mensaje. La forma de usarlo es:

Throw new TipoDeExcepción("mensaje") → Es de Tipo Nothing ¿Por qué? Porque Throw nunca devuelve un valor e interrumpe el flujo, luego Scala asigna Nothing cuando la expresión no Termina normalmente (lanza una excepción o hay ciclos infinitos)

Nothing es un Tipo especial, es subtipo de Todos los Tipos de Scala, se acepta en cualquier lado

• Try/Catch.

Es una forma de capturar excepciones y poder decidir qué hacer con ellas. La forma de usarlo es:

```
Try {
    //Código que puede producir una o más excepciones
}
catch {
   case e: TipoDeExcepción =>
        //qué hacer

   case e: TipoDeExcepción =>
        //qué hacer
}
```

> Podemos Tener más de un case

El Try/Catch es una expresión así que se puede guardar dentro de una variable

- Ejemplo de Throw y Try/Catch.

```scala
def dividir(a: Int, b: Int): Int = {
  if (b == 0) throw new ArithmeticException("No se puede dividir por cero")
  a / b
}

def main(args: Array[String]): Unit = {
  try {
    val resultado = dividir(10, 0)
    println(s"Resultado: $resultado")
  } catch {
    case e: ArithmeticException => println(s"Ocurrió un error: ${e.getMessage}")
  }

  println("El programa sigue ejecutándose normalmente")
}
```

↳ Reproduce el mensaje del Throw

- Try/Catch/Finally.

También podemos usar finally que nos permite poner código que se ejecute siempre (si termina bien, si se lanza una excepción, si no se captura la excepción, ...), la forma es:

```
Try {
    //Código que puede producir una o más excepciones
}
catch {
   case e: TipoDeExcepción =>
       //qué haces

   case e: TipoDeExcepción =>
       //qué haces
}
finally {
    //código que siempre se ejecuta
}
```

> Podemos tener más de un case

finally se usa para limpieza.

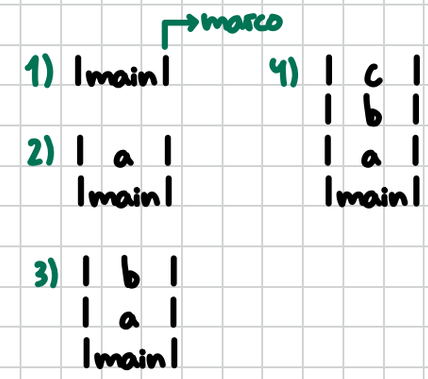
Recordar que el análisis es con el por Try/catch

Ej de Pila de llamadas y Heap donde se captura la excepción.

```scala
def c(): Unit = throw new ArithmeticException("error")
def b(): Unit = c()
def a(): Unit =
  try b()
  catch *
    case e: ArithmeticException => println("Atrapada en a")

a()
println("el programa sigue")
```

] main



5) En c se lanza la excepción.

new crea el objeto de la excepción en el heap

Por la forma del código, la excepción se lanza en el Try b() cuyo catch es *

Ahora se recorre la pila de arriba hacia abajo buscando un catch que capture la excepción

6) ¿c Tiene un catch? No, se quita el marco de c

Si c hubiera tenido un catch pero ningún case calza, es como si no tuviera catch

| b |
| a |
|main|

7) ¿b Tiene un catch? No, se quita el marco de b

| a |
|main|

8) ¿a Tiene un catch? Sí, la propagación se detiene, se ejecuta el case

| a |
|main|

9) El case imprime "Atrapada en a", a Termina, se quita su marco

|main|

10) Se sigue en main y se imprime "el programa sigue", main Termina, se quita su marco

Ojo, el main sí podría tener catch (main es un método más).

Si es que la pila queda sin marcos y aún no se encontró un catch que logre capturar la excepción, entonces en JVM aparece "Exception in Thread main"

- Otro ej de Pila de llamadas y Heap donde no se captura la excepción.

```scala
class BaseException extends Exception
class DerivedException extends BaseException

@main def example4(): Unit =
  try throw new DerivedException
  catch *
    case e: DerivedException => throw e
    case e: BaseException    => println("BaseException")
```

→ Así se llama el main

1) |example4|

2) En example4 se lanza la excepción

new crea el objeto de la excepción en el heap

Por la forma del código, la excepción se lanza en el Try Throw new DerivedException cuyo catch es *

Ahora se recorre la pila de arriba hacia abajo buscando un catch que capture la excepción

3) ¿example4 Tiene un catch? Sí, la propagación se detiene, se ejecuta el case

|example4|

4) El case vuelve a lanzar la excepción

Ahora se recorre la pila de arriba hacia abajo buscando un catch que capture la excepción

5) ¿example4 Tiene un catch? No, porque el Try/catch que hay es para el Throw new DerivedException, no es para el relanzamiento, se quita el marco de example4

6) La pila queda sin marcos y aún no se encontró un catch que logre capturar la excepción, entonces en JVM aparece "Exception in Thread main"

• Otro ej de Pila de llamadas y Heap donde se captura la excepción.

```scala
def hacerAlgo(): Unit =
  try throw new DerivedException
  catch *
    case e: DerivedException => throw e

@main def programa(): Unit =
  try hacerAlgo()
  catch
    case e: BaseException => println("Atrapada en programa")
```

1) |programa|

2) |hacerAlgo|
   |programa|

3) En hacerAlgo se lanza la excepción

new crea el objeto de la excepción en el heap

Por la forma del código, la excepción se lanza en el Try Throw new DerivedException cuyo catch es *

Ahora se recorre la pila de arriba hacia abajo buscando un catch que capture la excepción

3) ¿hacerAlgo Tiene un catch? Sí, la propagación se detiene, se ejecuta el case *

|hacerAlgo|
|programa|

4) El case vuelve a lanzar la excepción

Ahora se recorre la pila de arriba hacia abajo buscando un catch que capture la excepción

5) ¿hacerAlgo Tiene un catch? No, porque el Try/catch que hay es para el Throw new DerivedException, no es para el relanzamiento, se quita el marco de hacerAlgo

|programa|

6) ¿programa Tiene un catch? Sí, la propagación se detiene, se ejecuta el case

|programa|

7) El case imprime "Atrapada en programa", programa termina, se quita su marco

- Otro ej de Pila de llamadas y Heap donde se captura la excepción.

```scala
def hacerAlgo(): Unit =
  try                                          // try EXTERNO
    try throw new DerivedException             // try INTERNO
    catch *
      case e: DerivedException => throw e      // relanza
  catch *                                      // catch EXTERNO
    case e: BaseException => println("Atrapada por el catch externo")
```

1) |hacerAlgo|

3) En hacerAlgo se lanza la excepción

new crea el objeto de la excepción en el heap

Por la forma del código, la excepción se lanza en el Try Throw new DerivedException cuyo catch es *

Ahora se recorre la pila de arriba hacia abajo buscando un catch que capture la excepción

3) ¿hacerAlgo Tiene un catch? Sí, la propagación se detiene, se ejecuta el case

|hacerAlgo|

4) El case vuelve a lanzar la excepción

Por la forma del código, la excepción se lanza en el Try cuyo catch es *

Ahora se recorre la pila de arriba hacia abajo buscando un catch que capture la excepción

5) ¿ hacerAlgo Tiene un catch? Sí, la propagación se detiene, se ejecuta el case

|hacerAlgo|

6) El case imprime "Atrapada por el catch externo", programa Termina, se quita su marco

- Ej de función que recibe una función.

Trait y clases

```
trait Animal:
  def talk(): Animal

class Dog(name: String) extends Animal:
  override def talk(): Dog =
    println(s"$name says Woof")
    this → retorna el mismo objeto
```

```
class Cat(name: String) extends Animal:
  override def talk(): Cat =
    println(s"$name says Meow")
    this

  def meow(): Cat =
    println(s"$name is meowing")
    this
```

Funciones

```
def foo(f: Animal => Animal): Animal =
  f(new Dog("Bond")).talk()
def present(a: Animal): Animal =
  a.talk()
def doMeow(c: Cat): Cat =
  c.meow()
def objectify(d: Dog): Object =
  new Object
def doPrint(o: Object): Dog =
  println(o)
  new Dog("Bond")
```

Ejecutando foo(present)

```
foo(present)
-> present(new Dog("Bond")).talk()
// dog = new Dog("Bond")
-> present(dog).talk()
-> { dog.talk() }.talk()
-> { dog }.talk()  → Se imprime "Bond says woof" y se devuelve dog
-> dog.talk()
      ↳ Se imprime "Bond says woof" y se devuelve dog
```